Session02 - Câu lệnh khiển

Khái niệm

- Câu lệnh có quyền thay đổi luồng thực thi của chương trình
- Quyết định các câu lệnh khác được thực thi và thực thi khi nào

- Các câu lệnh điều khiển
  - **Condition Statement (Decision Marking)** – Câu lệnh điều kiện
  - **Iterative Statement (Loop)** – Câu lệnh lặp
  - **Jump Statement** – Câu lệnh nhảy

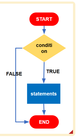
Câu lệnh điều kiện

Khái niệm

- Câu lệnh điều kiện dùng để ***thay đổi luồng thực thi của chương trình*** dựa theo một ***điều kiện*** nào đó

Câu lệnh IF

- Các mệnh đề
  - mệnh đề **if**



  - mệnh đề **else if**

  - mệnh đề **else**

- Các dạng cú pháp
  - Câu lệnh chỉ có if :

```
if(condition){
            // Statements in if
}
```

  - Câu lệnh if - else :

```
if (condition) {
            // Statements in if
} else {
            // Statements in else
}
```

  - Câu lệnh if-else if -else :

```
if ( condition ) {
            // Statements in if
} else if ( condition ) {
            // Statements in else if
} else {
            // Statements in else
}
```

  - Trong đó
    - ***condition*** là biểu thức điều kiện trả về giá trị kiểu boolean
    - ***Statements in if*** là các câu lệnh được thực thi nếu điều kiện trả về đúng, còn nếu sai thì sẽ thực hiện các câu lệnh ***Statements in else***

Câu lệnh Switch

- Cú pháp:

```
switch ( expression ) {
      case value1 :
        // Block Statements 1
        break ;
      case value2 :
        // Block Statements 2
        break ;
      . . . .
      case valueN :
        // Block Statements N
        break ;
      default :
        //Block Statements N+1
}
```

- Lưu ý :
  - **expression** là một biểu thức trả về giá trị dùng để so sánh với các trường hợp trong các case. Có thể nhận các kiểu dữ liệu như: ***byte, short, char, String, enum*** và các lớp ***Wrapper Types*** cho những kiểu trên
  - Nếu có trường hợp bằng nhau thì khối lệnh tương ứng sẽ được thực thi
  - Nếu không có trường hợp nào bằng nhau thì khối lệnh trong ***default*** (nếu có) sẽ được thực thi
- Lệnh **break** dùng trong câu lệnh **switch - case**
  - Nếu gặp lệnh **break** trong case thì sẽ dừng và thoát khỏi **switch** ngay lập tức
  - Nếu gặp trường hợp bằng nhau, nhưng sau đó không có câu lệnh **break** thì tất cả những khối lệnh phía sau cũng được thực thi

Ứng dụng cơ bản

- Câu lệnh **IF**
  - có thể dùng bất kỳ biểu thức nào trả về **boolean** để đưa ra quyết định.
  - Thích hợp với điều kiện phức tạp, dễ dàng (*có nhiều phép so sánh khác nhau, kết hợp với toán tử logic như* **&&, ||**)
- Câu lệnh **Switch**
  - chỉ so sánh giá trị của một biểu thức với một tập hợp các giá trị cố định (hỗ trợ kiểu ***int, char, String, enum, v.v.***).
  - Phù hợp với các điều kiện đơn giản, rõ ràng, khi bạn biết trước các giá trị cụ thể có.

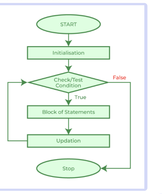
Câu lệnh lặp

Khái niệm

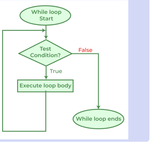
- **Loop - Vòng lặp**
  - Giúp bạn giảm nhiều câu lệnh được thực thi lặp đi lặp lại cho đến khi điều kiện vòng lặp được thỏa mãn

Phân loại

- Lặp với số lần lặp **BIẾT TRƯỚC**
  - For..loop
    - Cú pháp (Syntax)
      - **for(initialization; condition; updation)** {
        // **body**
        }
        - **Initialization** - Khởi tạo giá trị cho biến điều khiển vòng lặp
        - **Condition** - Điều kiện kiểm tra xem vòng lặp có được tiếp tục thực thi hay không
        - **Body** - Đoạn mã được lặp đi lặp lại bởi vòng lặp for
        - **Updation** - Cập nhật lại giá trị (Tăng/Giảm) biến điều khiển vòng lặp
      - **for(int** i = 1; i <= 10; i = i + 1) {
        System.out.println("Hello world");
        }
        - **Lặp lại** tác vụ in ra chuỗi **"Hello world" 10 lần**
    - Luồng hoạt động
      - Vòng lặp **for** hoạt động dựa trên một biến điều khiển. Thông thường biến điều khiển này được gọi với các chữ cái **i, j, k ...**
      - **Bước 1: Initialization** - Thiết lập giá trị khởi tạo cho biến điều khiển vòng lặp. Bước này chỉ xảy ra 1 lần duy nhất trong lần lặp đầu tiên của vòng lặp
      - **Bước 2: Condition** - Sau khi biến điều khiển vòng lặp (**i**) đã có giá trị, biến điều khiển này sẽ được kiểm tra với điều kiện **condition**.
        - Thông thường giá trị khởi tạo sẽ là các giá trị 0, 1 hoặc tuỳ thuộc vào tính huống của các bài toán trong thực tế
        - Nếu thoả mãn (true), đoạn mã trong phần **body** của vòng lặp mới được thực thi
        - Nếu không thoả mãn (false), vòng lặp ngay lập tức dừng lại
      - **Bước 3: Body** - Sau khi biến điều khiển vòng lặp đã thoả mãn điều kiện (**condition**) thì **đoạn mã trong body** của vòng lặp sẽ **được thực thi**
      - **Bước 4: Updation** - Cập nhật lại giá trị (Tăng/Giảm) biến điều khiển vòng lặp
      - **Bước 5: Repetition** - Sau khi biến điều khiển vòng lặp được cập nhật lại giá trị, vòng lặp quay lại **Bước 2** và lặp đi lặp lại cho đến khi điều kiện **condition** không thoả mãn (**false**) thì sẽ **dừng lại**
    - Lưu đồ (Flowchart)



  - While..loop
    - Cú pháp (Syntax)
      - **while(condition)** {
        // **body**
        }
        - **Condition** - Điều kiện kiểm tra xem vòng lặp có được tiếp tục thực thi hay không
        - **Body** - Đoạn mã được lặp đi lặp lại bởi vòng lặp while
      - **int input** = -1;
        Scanner sc = new **Scanner(System.in)**;
        while(answer != 0) {
        System.out.println("Hãy nhập 1 số nguyên : ");
        input = sc.nextInt();
        }
        System.out.println("Chúc mừng bạn đã hoàn thành !!!");
        - Vòng lặp được thực hiện cho đến khi người dùng nhập vào số 0
    - Luồng hoạt động
      - Vòng lặp **while** hoạt động dựa trên một điều kiện lặp (**condition**) nhất định
      - **Bước 1: Condition** - Điều kiện khởi tạo để vòng lặp while có thể hoạt động được;
        - Nếu điều kiện thoả mãn (true), đoạn mã trong phần **body** của vòng lặp mới được thực thi
        - Nếu điều kiện không thoả mãn (false), vòng lặp ngay lập tức dừng lại
      - **Bước 2: Body** - Sau khi điều kiện điều khiển vòng lặp while (**condition**) được thoả mãn (**true**), **đoạn mã trong body** của vòng lặp sẽ **được thực thi**
      - **Bước 3: Repetition** - Vòng lặp tiếp tục quay lại **Bước 1** và lặp đi lặp lại cho đến khi điều kiện **condition** không thoả mãn (**false**) thì sẽ **dừng lại**
    - Lưu đồ (Flowchart)



    - Lưu ý
      - Vòng lặp vô hạn (**Infite loop**) xảy ra khi điều kiện lặp (**condition**) mãi mãi luôn đúng.
      - Cần chú ý trong việc thiết lập điều kiện lặp cho vòng lặp **while**
      - Điều kiện lặp (**condition**) thường liên quan trực tiếp đến bài toán đang cần giải quyết trong thực tế
      - Có thể tạo ra 1 **biến** dùng để **kiểm soát vòng lặp while**
      - **Biến kiểm soát vòng lặp** sẽ được dùng để **thiết lập điều kiện lặp**
      - Tuỳ thuộc vào các tình huống của bài toán trong thực tế, **cập nhật lại giá trị của biến kiểm soát vòng lặp** (để **condition không còn thoả mãn** -> **Vòng lặp dừng lại**
      - **int** input = -1;
        boolean loop = true;
        Scanner sc = new **Scanner(System.in)**;
        while(loop) {
        System.out.println("Hãy nhập 1 số nguyên : ");
        input = sc.nextInt();
        if (input == 0) {
        loop = false;
        }
        }
        System.out.println("Chúc mừng bạn đã hoàn thành !!!");
    - Hướng tiếp cận khác
      - Vòng lặp vô hạn (**Infite loop**)
      - Điều kiện lặp (**Condition**)
  - Do-while..loop
    - Cú pháp (Syntax)
      - **do** {
        // **body**
        } **while(condition)**;
      - **int** answer;
        Scanner sc = new Scanner(System.in);
        do {
        System.out.println("Hãy nhập 1 số nguyên : ");
        answer = sc.nextInt();
        } **while(answer** != 10)
        System.out.println("Chúc mừng bạn đã hoàn thành !!!");
        - Vòng lặp sẽ thực hiện việc nhập đến khi người dùng nhập vào giá trị 10 thì dừng lại
    - So sánh
      - Vòng lặp **do...while** có luồng hoạt động giống với vòng lặp **while**
      - Vì **do** được xử lý trước khi **condition** được kiểm tra nên vòng lặp **do...while** được thực thi **ít nhất 1 lần** kể cả khi **condition** có **không thoả mãn (false) ngay từ lần lặp đầu tiên**.
- Lặp với số lần lặp **KHÔNG BIẾT TRƯỚC**
  - break
    - Từ khoá dùng để ngay lập tức dừng vòng lặp và thoát ra khỏi vòng lặp tại bất kỳ thời điểm nào
    - **for(int** i = 1; i <= 10; i = i + 1) {
      System.out.println("Vòng lặp thứ " +i);
      if (i == 3) {
      break;
      }
      }
      - Vòng lặp dừng lại tại thời điểm giá trị của biến kiểm soát vòng lặp **i == 3**
  - continue
    - Từ khoá dùng để dừng vòng lặp tại lần lặp hiện tại và chuyển đến lần lặp tiếp theo
    - Toàn bộ đoạn mã dưới từ khoá **continue** sẽ bị **dừng lại (Không thực thi)** và ngay lập tức vòng lặp sẽ **chuyển đến lần lặp tiếp theo**
    - **for(int** i = 1; i <= 10; i = i + 1) {
      if (i == 3) {
      continue;
      }
      System.out.println("Vòng lặp thứ " +i);
      - Vòng lặp **sẽ bỏ qua** toàn bộ logic trong body tại thời điểm **i == 3**
  - return
    - Lệnh này dùng để dừng hàm và trả về giá trị , cũng sẽ kết thúc vòng lặp

Presented with xmind